

Below are **senior-level .NET Core + Angular scenario-based interview questions focused on optimization, scalability, debugging, API performance, database performance, Angular rendering, memory management, caching, and production issues**. These are the kinds of scenarios commonly used in MNC interviews for Senior Developer/Lead roles.

I'll explain not only the answer, but also **how to approach the problem in an interview**.

1. .NET Core API is taking 8–10 seconds. How will you identify the performance issue?

Scenario

An Angular page calls:

```
http
GET /api/orders/12345
```

The API takes around **8 seconds**.

The interviewer asks:

How will you identify whether the issue is Angular, API, database, network, or external service?

Answer

I would not immediately change the code. First, I would identify **where the time is being spent**.

The request flow may look like:

```
Angular
  ↓
API Gateway
  ↓
ASP.NET Core API
  ↓
Business Service
  ↓
Repository
  ↓
SQL Server
```

↓
External APTs

I would measure every layer.

For example:

C#

```
var stopwatch = Stopwatch.StartNew();

var customer = await _customerRepository.GetCustomer(id);

_logger.LogInformation(
    "Customer DB query completed in {Elapsed}ms",
    stopwatch.ElapsedMilliseconds);
```

Then similarly measure external API calls.

Things to check

1. Browser DevTools → Network
2. API response time
3. SQL execution time
4. EF Core generated query
5. External API latency
6. CPU utilization
7. memory utilization
8. thread pool starvation
9. synchronous blocking calls
10. excessive serialization
11. large payload
12. repeated database calls

For production monitoring, tools could include:

Application Insights
OpenTelemetry
Serilog
SQL Profiler
SQL Extended Events
Azure Monitor
Grafana
New Relic
Dynatrace

Strong interview answer

I first measure instead of guessing. I divide the request into frontend time, network time, API processing time, database time, and external dependency time. Once I identify the bottleneck, I optimize that particular layer.

2. Angular page takes 10 seconds to load because it calls 8 APIs. How will you optimize it?

Suppose Angular does:

TypeScript

```
ngOnInit() {  
  this.loadCustomer();  
  this.loadOrders();  
  this.loadProducts();  
  this.loadAddress();  
  this.loadPayment();  
  this.loadHistory();  
  this.loadDocuments();  
  this.loadNotifications();  
}
```

This creates many network calls.

One solution is parallel loading.

TypeScript

```
forkJoin({  
  customer: this.customerService.getCustomer(),  
  orders: this.orderService.getOrders(),  
  products: this.productService.getProducts(),  
  notifications: this.notificationService.getNotifications()  
}).subscribe(result => {  
  
});
```

But sometimes a better architecture is a backend aggregation API.

Instead of:

Angular

↓

8 HTTP calls

create:

http

GET /api/dashboard

Backend:

C#

```
public async Task<DashboardDto> GetDashboard()
{
    var customerTask = _customerService.GetCustomer();
    var ordersTask = _orderService.GetOrders();
    var notificationsTask = _notificationService.GetNotifications();

    await Task.WhenAll(
        customerTask,
        ordersTask,
        notificationsTask);

    return new DashboardDto
    {
        Customer = await customerTask,
        Orders = await ordersTask,
        Notifications = await notificationsTask
    };
}
```

Now:

Angular

↓

Dashboard API

↓

Customer Orders Notifications

This is often called:

Backend for Frontend — BFF

3. Your API fetches 100,000 records from SQL Server and Angular becomes slow. What will you do?

Bad implementation:

C#

```
var employees = await _context.Employees.ToListAsync();
```

Returning 100,000 records causes:

```
Database load
    ↓
API memory load
    ↓
JSON serialization
    ↓
Network transfer
    ↓
Angular memory
    ↓
DOM rendering
```

So optimization should happen at multiple levels.

Database pagination

C#

```
var employees = await _context.Employees
    .AsNoTracking()
    .OrderBy(x => x.Id)
    .Skip((pageNumber - 1) * pageSize)
    .Take(pageSize)
    .Select(x => new EmployeeDto
    {
        Id = x.Id,
        Name = x.Name,
        Department = x.Department
    })
    .ToListAsync();
```

API:

http

```
GET /api/employees?page=1&pageSize=50
```

Response:

JSON

```
{
  "totalCount": 100000,
```

```
"pageNumber": 1,  
"pageSize": 50,  
"items": []  
}
```

Angular displays only the requested records.

Important

Don't do:

```
C#  
  
var employees = await _context.Employees.ToListAsync();  
  
return employees  
    .Skip(...)  
    .Take(...);
```

because the filtering occurs in memory.

The SQL query should contain the pagination.

4. Your EF Core API is consuming a lot of memory. How will you optimize it?

One common reason is EF Core **change tracking**.

Example:

```
C#  
  
var users = await _context.Users.ToListAsync();
```

EF tracks every entity.

If this API is read-only:

```
C#  
  
var users = await _context.Users  
    .AsNoTracking()  
    .ToListAsync();
```

This reduces tracking overhead.

Better:

C#

```
var users = await _context.Users
    .AsNoTracking()
    .Select(x => new UserDto
    {
        Id = x.Id,
        Name = x.Name
    })
    .ToListAsync();
```

Avoid returning entire entities.

Bad:

C#

```
return await _context.Users
    .Include(x => x.Department)
    .Include(x => x.Roles)
    .Include(x => x.Permissions)
    .Include(x => x.Orders)
    .ToListAsync();
```

If the UI needs only:

```
Id
Name
Department
```

select only those.

5. An API performs three independent operations. It takes six seconds. How will you improve it?

Example:

C#

```
var user = await GetUser();
var orders = await GetOrders();
var notifications = await GetNotifications();
```

Execution:

GetUser	2 sec
---------	-------

GetOrders	2 sec
GetNotifications	2 sec

Total	6 sec

If operations are independent:

C#

```
var userTask = GetUser();
var ordersTask = GetOrders();
var notificationTask = GetNotifications();

await Task.WhenAll(
    userTask,
    ordersTask,
    notificationTask);
```

Now approximately:

User	—————	2 sec
Orders	—————	2 sec
Notification	——	2 sec
Total ≈ 2 sec		

However, I would also mention:

Parallelize only independent I/O operations. Don't blindly parallelize database queries using the same EF Core DbContext because DbContext is not thread-safe.

That is a strong senior-level point.

6. Why can `.Result` and `.Wait()` cause performance problems in ASP.NET Core?

Bad:

C#

```
var result = _service.GetDataAsync().Result;
```

or:

C#

```
_service.GetDataAsync().Wait();
```

Better:

C#

```
var result = await _service.GetDataAsync();
```

Although classic ASP.NET Core doesn't have the same synchronization-context deadlock behavior as older ASP.NET, blocking still creates problems.

`.Result` consumes a worker thread waiting for I/O.

Imagine:

1000 concurrent requests

Each blocks a thread:

```
Thread 1 → waiting SQL  
Thread 2 → waiting API  
Thread 3 → waiting API  
...
```

Eventually:

```
Thread Pool Starvation  
↓  
Request Queue  
↓  
Latency increases  
↓  
Timeouts
```

With async:

```
Request  
↓  
await database  
↓  
Thread returned to pool  
↓  
I/O completes  
↓  
Continuation executes
```

Therefore:

Async/await improves scalability primarily by freeing request threads while waiting for I/O.

7. Your database query takes five seconds. What steps will you take?

I would investigate:

```
SQL query
Execution plan
Indexes
Table scan
Key lookup
Joins
Statistics
Blocking
Parameter sniffing
Returned columns
Returned rows
```

Example bad query:

SQL

```
SELECT *
FROM Orders
WHERE CustomerId = 1001;
```

Possible index:

SQL

```
CREATE INDEX IX_Orders_CustomerId
ON Orders (CustomerId);
```

Better API projection:

C#

```
var orders = await _context.Orders
    .Where(x => x.CustomerId == customerId)
    .Select(x => new OrderDto
    {
        Id = x.Id,
        OrderDate = x.OrderDate,
        Amount = x.Amount
    })
```

```
    })  
    .AsNoTracking()  
    .ToListAsync();
```

Strong interview response:

I would examine the actual SQL generated by EF Core and analyze the SQL execution plan before changing application code.

8. Explain the N+1 query problem in EF Core.

Suppose:

C#

```
var customers = await _context.Customers.ToListAsync();  
  
foreach (var customer in customers)  
{  
    var orders = customer.Orders;  
}
```

With lazy loading, it may generate:

Query 1 → Customers

Query 2 → Customer 1 orders

Query 3 → Customer 2 orders

Query 4 → Customer 3 orders

...

Query 101

That's the N+1 problem.

Instead:

C#

```
var customers = await _context.Customers  
    .Select(x => new CustomerDto  
    {  
        Id = x.Id,  
        Name = x.Name,  
  
        Orders = x.Orders.Select(o => new OrderDto  
        {  
            Id = o.Id,
```

```
        Amount = o.Amount
    }).ToList()
})
.ToListAsync();
```

Depending on the scenario, also consider:

C#

```
Include()
```

or split queries.

9. Angular table containing 10,000 rows is very slow. What will you do?

Rendering 10,000 DOM rows is expensive.

Bad:

HTML

```
<tr *ngFor="let employee of employees">
```

Possible optimizations:

Server-side pagination

Return:

50-100 records

per request.

TrackBy

Traditional example:

TypeScript

```
trackByEmployeeId(index: number, item: Employee) {
    return item.id;
}
```

HTML

```
<tr *ngFor="let employee of employees; trackBy: trackByEmployeeId">
```

With newer Angular syntax:

HTML

```
@for(employee of employees; track employee.id) {  
  ...  
}
```

Angular can then identify which row actually changed.

Without tracking, unnecessary DOM operations can occur.

Also consider:

```
Virtual scrolling  
Lazy loading  
Server-side filtering  
Server-side sorting  
Server-side pagination  
OnPush / Signals
```

10. How does Angular `OnPush` improve performance?

Default change detection can cause large component trees to be checked frequently.

Example:

TypeScript

```
@Component({  
  selector: 'app-order',  
  changeDetection: ChangeDetectionStrategy.OnPush  
})  
  
export class OrderComponent {}
```

With `OnPush` Angular can avoid unnecessary checks and update the component mainly when relevant input references/events/reactive state change.

For modern Angular applications I would combine:

```
OnPush  
Signals  
Immutable state  
track
```

Example:

TypeScript

```
orders = signal<Order[]>([]);

total = computed(() =>
  this.orders().reduce((a, b) => a + b.amount, 0)
);
```

11. Angular application gradually becomes slow after 30 minutes of usage. What do you suspect?

A common suspect is:

memory leak

For example:

TypeScript

```
ngOnInit() {
  this.service.getUpdates().subscribe(x => {
    this.data = x;
  });
}
```

If that Observable keeps emitting and the component is repeatedly created/destroyed, subscriptions can accumulate.

Modern Angular:

TypeScript

```
private destroyRef = inject(DestroyRef);

ngOnInit() {

  this.service.getUpdates()
    .pipe(takeUntilDestroyed(this.destroyRef))
    .subscribe(result => {
      this.data = result;
    });
}
```

Or use:

HTML

```
{{ users$ | async }}
```

AsyncPipe handles subscription lifecycle automatically.

Memory leaks can also come from:

```
setInterval  
setTimeout  
DOM event listeners  
WebSocket subscriptions  
SignalR connections  
large cached arrays  
global services  
unreleased references
```

12. Angular search sends an API request for every keystroke. How will you optimize it?

Bad:

```
a → API  
ap → API  
app → API  
appl → API  
apple → API
```

That's five calls.

Use:

TypeScript

```
this.searchControl.valueChanges  
  .pipe(  
    debounceTime(300),  
    distinctUntilChanged(),  
    switchMap(value =>  
      this.productService.search(value)  
    )  
  )  
  .subscribe();
```

Now:

```
User types "apple"

wait 300ms

API → apple
```

Why `switchMap` ?

Suppose:

```
Search "app" → request 1
Search "apple" → request 2
```

If request 1 is still running, `switchMap` unsubscribes from the previous inner observable so its result won't incorrectly overwrite the newer search result.

This avoids race-condition-style UI behavior.

13. API returns the same master data thousands of times. How can you optimize it?

Suppose Angular repeatedly calls:

```
http

GET /api/countries
GET /api/departments
GET /api/status
```

These values rarely change.

Use caching.

Example:

```
C#

public async Task<IEnumerable<CountryDto>> GetCountries ()
{
    return await _cache.GetOrCreateAsync (
        "countries",
        async entry =>
        {
            entry.AbsoluteExpirationRelativeToNow =
```

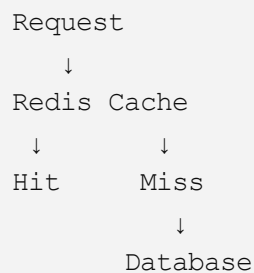
```
        TimeSpan.FromHours(1);

        return await _repository.GetCountries();
    });
}
```

For distributed applications:

Redis

Architecture:



Interview question

Should everything be cached?

No.

Avoid blindly caching:

- highly transactional data
- sensitive user-specific data without appropriate isolation
- very frequently changing data
- huge objects
- low-reuse data

14. When would you use MemoryCache vs Redis?

IMemoryCache

- Single instance
- Very fast
- Stored in application memory
- Lost when application restarts
- Not shared between servers

Example:

API Server 1 cache

Redis

Distributed
Shared across instances
Survives individual API process restart
Useful for web farms/microservices

Example:

```
      Redis
     /    \
API Server 1  API Server 2
```

For load-balanced enterprise applications:

Redis is usually preferable for shared caching.

15. Your Angular bundle is very large and first load is slow. How do you optimize it?

Suppose:

main.js = 8 MB

The first page load will be slow.

Use lazy loading.

```
TypeScript

{
  path: 'admin',
  loadChildren: () =>
    import('./admin/admin.component')
      .then(m => m.AdminComponent)
}
```

Instead of loading:

Login
Dashboard

Admin
Reports
Billing
Settings

at startup, load:

Initial

Login
Dashboard

and later:

Admin chunk
Reports chunk
Billing chunk

Also inspect:

unused dependencies
heavy chart libraries
large images
source maps
duplicate libraries
tree-shaking
production build configuration

16. A report API times out because it processes 1 million records. What architecture would you use?

Don't keep a web request open for a long-running operation.

Bad:

Angular
↓
HTTP Request
↓
Process 1M records
↓
60 seconds

Better:

```
Angular
  ↓
POST /reports
  ↓
Create Job
  ↓
Queue
  ↓
Background Worker
  ↓
Generate Report
  ↓
Blob Storage
```

API returns:

```
JSON

{
  "jobId": "AB123",
  "status": "Processing"
}
```

Angular checks:

```
http

GET /reports/AB123/status
```

Or use SignalR/WebSocket notification.

Possible technologies:

```
Azure Service Bus
RabbitMQ
Hangfire
BackgroundService
Azure Functions
Kafka
```

17. Your API CPU reaches 100%. How do you investigate?

First identify whether workload is:

CPU-bound

or

I/O-bound

CPU-heavy examples:

PDF generation
image manipulation
encryption
large loops
JSON processing
compression
complex calculations

I/O examples:

database
HTTP API
file system
blob storage

If CPU-bound:

profile code
remove inefficient loops
reduce allocations
cache reusable computations
move expensive work to background workers
scale horizontally

Do not assume `async` solves CPU-bound operations.

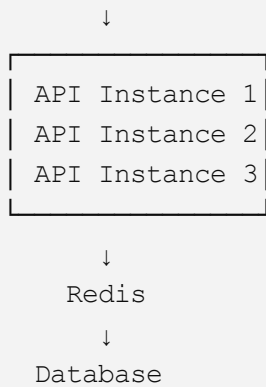
Async mainly helps:

I/O-bound scalability

18. API receives 5000 requests per second. How would you design it?

I would consider:

Load Balancer



Optimization areas include:

- stateless APIs
- horizontal scaling
- distributed cache
- database indexing
- async I/O
- connection pooling
- rate limiting
- pagination
- message queues
- CDN for static resources
- output caching
- response compression
- observability
- resiliency policies

19. External API takes 15 seconds. Your API is becoming slow. What will you do?

Don't simply increase timeout immediately.

First consider:

```
Timeout  
Retry  
Circuit Breaker  
Fallback  
Cache  
Bulkhead  
Background processing
```

Example resilience strategy:

```
Your API
  ↓
External API
  ↓
Failure
  ↓
Retry
  ↓
Failure
  ↓
Circuit Breaker Opens
```

With modern .NET resilience handling, these concepts can be implemented via resilient HTTP pipelines.

Important interview point:

Retries should normally be used only for transient failures, and retry policies should use delays/backoff. Blind retries can amplify an outage.

20. Your API returns 20 MB JSON. How would you optimize it?

I would check why such a large response is necessary.

Options:

```
Pagination
DTO projection
Filtering
Field selection
Compression
Streaming
Avoid Base64 blobs
Separate file download endpoints
```

Instead of:

```
JSON

{
  "employeeId": 1,
  "employeeName": "...",
  "department": {...},
  "history": [...],
```

```
"documents": [...],  
"permissions": [...]  
}
```

Return only required UI fields.

For example:

C#

```
.Select(x => new EmployeeListDto  
{  
    Id = x.Id,  
    Name = x.Name,  
    DepartmentName = x.Department.Name  
})
```

21. Angular page calls the same API multiple times. Why can this happen?

Suppose:

TypeScript

```
users$ = this.service.getUsers();
```

and template:

HTML

```
<div>{{ users$ | async }}</div>
```

```
<div>{{ users$ | async }}</div>
```

Depending on the observable, multiple subscriptions can cause multiple HTTP calls.

One solution:

TypeScript

```
users$ = this.service  
    .getUsers()  
    .pipe(  
        shareReplay({  
            bufferSize: 1,  
            refCount: true  
        })  
    )
```

```
);
```

But don't blindly cache every HTTP stream. Think about:

```
data freshness
cache invalidation
memory lifetime
error behavior
```

22. An API performs 100 database INSERTs inside a loop. How do you optimize it?

Bad:

```
C#

foreach (var employee in employees)
{
    _context.Employees.Add(employee);
    await _context.SaveChangesAsync();
}
```

That may result in many database round trips.

Better:

```
C#

_context.Employees.AddRange(employees);

await _context.SaveChangesAsync();
```

For very large imports consider:

```
SqlBulkCopy
bulk insert library
staging table
stored procedure / TVP
queue-based processing
batch processing
```

23. EF Core `Include()` made the API slower. Why?

Suppose:

C#

```
_context.Customers  
    .Include(x => x.Orders)  
    .Include(x => x.Addresses)  
    .Include(x => x.Payments)
```

This may produce very large join results.

Potential problem:

Cartesian explosion

Example:

```
1 Customer  
100 Orders  
20 Addresses  
30 Payments
```

The result set can multiply dramatically.

Depending on the case:

C#

```
.AsSplitQuery()
```

may help.

But usually the stronger solution is:

C#

```
Select(...)
```

only the data required by the UI.

24. Angular dashboard has 20 components and every action causes excessive re-rendering. How will you optimize it?

Investigate:

Change detection

Unnecessary subscriptions
Mutable objects
Expensive template functions
Large DOM
Repeated API calls

Avoid expensive methods directly in templates.

Bad:

HTML

```
<div>{{ calculateTotal() }}</div>
```

If `calculateTotal()` is expensive, it may be repeatedly invoked.

Prefer:

TypeScript

```
total = computed(() =>
  this.orders().reduce(
    (sum, order) => sum + order.amount,
    0
  )
);
```

Then:

HTML

```
{{ total() }}
```

For appropriate components, use:

Signals
`computed()`
`OnPush`
`track`
lazy-loaded components

25. SQL performance is good locally but very slow in production. Why?

Possible reasons include:

- Production data volume
- Missing indexes
- Different statistics
- Parameter sniffing
- Blocking/deadlocks
- Different execution plans
- Network latency
- CPU pressure
- Memory pressure
- Disk I/O
- Connection pool exhaustion
- Concurrent workload

Don't say:

"The query is working fine locally, so the code is fine."

Production workload and data distribution are fundamentally different.

Instead compare:

- local execution plan
- production execution plan
- local dataset
- production dataset
- local concurrency
- production concurrency

26. API performance becomes worse when 500 users log in simultaneously. What could be happening?

This is a classic scalability question.

Potential reasons:

- DB connection pool exhaustion
- Thread pool starvation
- Lock contention
- CPU saturation
- Memory pressure
- External API throttling
- Synchronous code
- Large session state
- Cache stampede

One important issue is **cache stampede**.

Suppose a cache expires at 10:00 AM.

500 users simultaneously request the same value.

```
500 requests
    ↓
Cache expired
    ↓
500 DB queries
```

Instead implement strategies such as:

```
distributed locking
single-flight cache refresh
staggered expirations
background cache refresh
```

27. What is connection pooling and why is it important?

Creating a database connection is expensive.

Connection pooling maintains reusable connections.

```
API Request
    ↓
Connection Pool
    ↓
Existing SQL Connection
```

But you should still dispose connections properly.

With EF:

```
C#

await using var context = ...
```

Normally DI manages DbContext lifetime.

A common registration:

C#

```
services.AddDbContext<AppDbContext>(
    options =>
        options.UseSqlServer(connectionString));
```

DbContext is normally:

Scoped per request

Do not use a singleton DbContext.

28. Angular gets data quickly, but UI freezes for five seconds. Where is the problem?

If:

```
Network = 200 ms
API = 150 ms
UI rendering = 5 sec
```

the backend isn't the bottleneck.

Investigate:

```
large DOM
heavy loops
large JSON transformation
complex change detection
expensive calculations
chart rendering
sorting/filtering huge arrays
```

Example:

TypeScript

```
this.users = result
    .filter(...)
    .map(...)
    .sort(...)
```

on hundreds of thousands of objects can freeze the browser.

Solutions:

server-side processing
pagination
virtualization
Web Workers for CPU-heavy browser tasks
memoized/computed state
smaller datasets

29. When should filtering happen in Angular and when on the server?

For small datasets:

Angular filtering can be fine

For very large datasets:

server-side filtering

Example:

http

```
GET /api/users?  
page=1  
&pageSize=50  
&search=venkat  
&department=IT  
&sortBy=name
```

SQL:

SQL

```
WHERE Name LIKE @Search  
AND Department = @Department  
ORDER BY Name  
OFFSET ...  
FETCH NEXT ...
```

Avoid:

DB 1,000,000 records

↓

API

↓

```
Angular
  ↓
filter()
```

30. A request occasionally takes 30 seconds but normally takes one second. How will you troubleshoot?

This is a strong MNC production support scenario.

I would correlate traces using a:

```
Correlation ID / Trace ID
```

For example:

```
Request ID: 9f23-df32
```

Then trace:

```
Angular
  ↓
API Gateway
  ↓
Service A
  ↓
Service B
  ↓
Database
```

Using distributed tracing:

```
OpenTelemetry
Application Insights
Jaeger
Zipkin
```

I would check:

```
SQL blocking
external API latency
GC pause
CPU spike
thread starvation
connection pool exhaustion
cache expiry
```

network latency
deployment event

31. Angular application initially loads slowly, but after refresh it is fast. Why?

Possible reasons:

Browser cache
JS bundle cache
API cache
DNS/TLS connection reuse
lazy chunk caching
service worker
backend cold start
JIT/warm-up

You should compare:

Cold load
vs
Warm load

Browser DevTools can disable cache to reproduce first-load behavior.

32. API is making the same reference-data DB query for every request. What pattern would you use?

Use **Cache-Aside Pattern**.

```
graph TD
    Application --> CheckCache[Check Cache]
    CheckCache --> FoundReturn[Found? → Return]
    FoundReturn --> NotFound[Not found]
    NotFound --> Database[Database]
    Database --> PutInCache[Put in Cache]
    PutInCache --> Return[Return]
```

Pseudo-code:

```
C#  
  
var data = await cache.GetAsync(key);  
  
if (data != null)  
    return data;  
  
data = await repository.GetData();  
  
await cache.SetAsync(key, data);  
  
return data;
```

The difficult part isn't adding caching.

The difficult part is:

Cache invalidation.

33. User complains that API becomes slow only at 9 AM every morning. What do you investigate?

This tests production thinking.

Possible scheduled workload:

```
Batch jobs  
Reports  
database maintenance  
ETL  
Backups  
cache expiration  
schedulers  
Windows services  
Hangfire jobs  
Azure Functions  
cron jobs
```

Look at metrics around the exact time:

```
CPU  
Memory  
Disk  
SQL waits
```

Blocked sessions
API latency
Request rate
Queue length

34. Your microservice architecture performs poorly because one request calls six services sequentially.

Current design:

```
Angular
  ↓
API Gateway
  ↓
Service A
  ↓
Service B
  ↓
Service C
  ↓
Service D
  ↓
Database
```

This creates latency amplification.

Example:

```
A = 200ms
B = 300ms
C = 400ms
D = 500ms

Total > 1.4 seconds
```

Solutions depend on the business process:

```
Parallelize independent calls
Cache reference data
BFF/API aggregation
Event-driven architecture
Denormalized read model
CQRS
Reduce chatty microservices
```

Strong interview observation:

Microservices are not automatically faster. Excessive synchronous inter-service communication can make performance worse than a modular monolith.

35. Your SQL query is fast but the API is still slow. What else will you check?

Many candidates stop at SQL.

But the request consists of:

```
Controller
Middleware
Authentication
Authorization
Business logic
Database
Serialization
Network
Client rendering
```

I would measure:

```
Authentication latency
Middleware latency
Mapping
JSON serialization
Logging
Encryption
External APIs
Blob access
Network transfer
Angular rendering
```

36. Angular application calls API every time user navigates back to the screen. Should we cache the data?

It depends on the business requirement.

For relatively static data:

TypeScript

```
private customers = signal<Customer[]>([]);

loadCustomers() {

    if(this.customers().length > 0)
        return;

    this.api.getCustomers()
        .subscribe(data =>
            this.customers.set(data)
        );
}
```

However, for transactional screens, stale data may be unacceptable.

Possible approaches:

```
cache + TTL
cache + explicit refresh
invalidate after update
stale-while-revalidate
no caching for critical live data
```

This shows you understand that caching is not only a technical choice—it depends on data consistency requirements.

37. How would you optimize a complete .NET Core + Angular application?

For an interview, you can organize your answer into **six layers**:

1. Angular
2. Network
3. API
4. Database
5. Caching
6. Infrastructure

Angular

```
Lazy loading
Signals
OnPush
```

- track
- server pagination
- debounce search
- virtualization
- avoid memory leaks
- reduce bundle size

Network

- HTTP compression
- smaller JSON
- CDN
- HTTP/2 / HTTP/3 where applicable
- browser caching

ASP.NET Core

- Async/await
- DTO projection
- response/output caching where safe
- parallel independent I/O
- background jobs
- connection pooling
- rate limiting
- efficient serialization

EF Core / SQL Server

- AsNoTracking
- Select projection
- indexes
- avoid N+1
- pagination
- query optimization
- execution plans
- bulk operations

Caching

- IMemoryCache
- Redis
- cache-aside
- cache invalidation

Infrastructure

Load balancing
horizontal scaling
CDN
Redis
message queue
monitoring
autoscaling

38. Interviewer: "Our production API is slow. What is your first action?"

A strong senior answer is:

I would not start changing code immediately. First I would establish a performance baseline and use telemetry to identify where the latency occurs.

Then:

```
Measure
  ↓
Identify bottleneck
  ↓
Form hypothesis
  ↓
Optimize
  ↓
Load test
  ↓
Compare before/after
  ↓
Deploy
  ↓
Monitor
```

This is much stronger than saying:

"I'll add caching."

Caching may hide the actual issue.

39. Scenario: API works fine for 10 users but crashes for 1000 users. How will you solve it?

This is where **load testing** becomes important.

Use tools such as:

```
k6
JMeter
Azure Load Testing
NBomber
```

Test:

```
100 users
500 users
1000 users
5000 users
```

Measure:

```
P50 latency
P95 latency
P99 latency
throughput
error rate
CPU
memory
DB connections
thread pool
GC
```

For example:

```
P50 = 120ms
P95 = 900ms
P99 = 5sec
```

Average alone can hide bad tail latency.

40. How would you explain your performance optimization process to an interviewer?

Use this framework:

```
M → Measure
I → Identify
O → Optimize
V → Validate
M → Monitor
```

Example answer

Suppose an API takes:

5 seconds

Telemetry shows:

API logic	100ms
Database	4.2 sec
Serialization	200ms
Network	500ms

I focus on:

Database

Execution plan shows:

Table Scan

I introduce an appropriate index and improve the query.

After optimization:

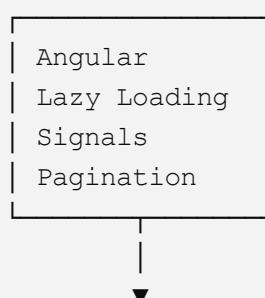
Database	200ms
API total	500ms

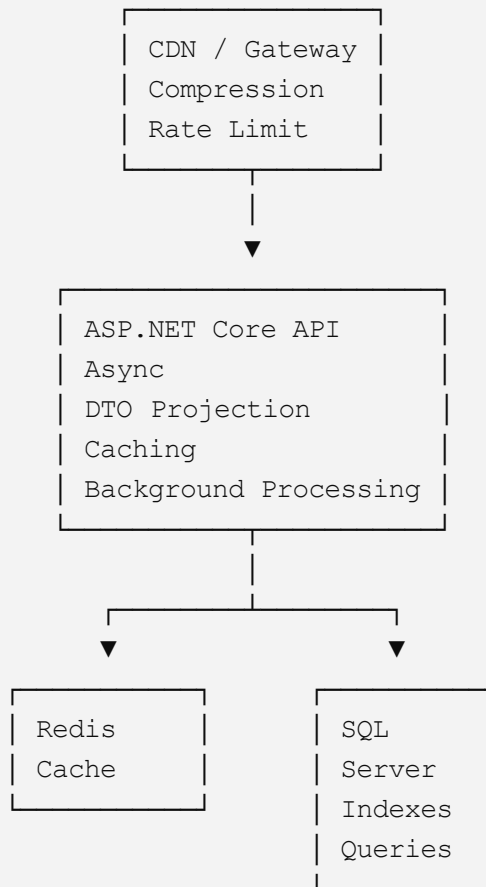
Then I run load tests and monitor the production metrics.

That is the kind of explanation expected from a **Senior Developer or Technical Lead**, because it demonstrates that you optimize based on evidence rather than assumptions.

One architecture answer worth remembering

For many MNC optimization questions, you can explain the flow like this:





A strong interview summary would be:

"For .NET Core + Angular performance problems, I optimize end-to-end. On Angular I focus on bundle size, lazy loading, change detection, Signals, request handling, and DOM rendering. On the API I focus on async I/O, DTO projection, caching, serialization, background jobs, and resiliency. At the data layer I focus on indexes, query plans, pagination, N+1 queries, EF tracking, and connection management. Finally, I validate improvements using load testing and production telemetry such as P95/P99 latency rather than relying only on average response time."

Since you're preparing for experienced .NET interviews, these **production troubleshooting explanations** are more valuable than definition-based answers because interviewers usually follow up with *"How do you know?"*, *"What if it doesn't work?"*, and *"How would this behave with 1,000 users?"*.